

Webサイト開発勉強会

オリジナルの自己紹介サイトを作ろう！

今日のゴール

自分の自己紹介ページを作る

完成するとこんなページができます 📌

- 名前や写真を載せたページを作れる
- 色や配置を変えて、自分らしく見せられる
- SNS などへのリンクを置ける

田中 はなこ

Web開発をはじめたばかりの大学生です！



こんにちは！田中はなこです。
東京に住んでいて、カフェ巡りと猫が好きです。最近プログラミングを始めました！

好きなもの

- カフェ巡り
- 猫
- ラーメン

リンク

jig.jp

小山高専 HP

今日の授業が終わるとできること

- HTMLで「ページに何を置くか」を自分で決められる
- CSSで「見やすさ」や「雰囲気」を変えられる
- 1枚の自己紹介ページを自力で作れる

今日覚えるのは細かい暗記より、
「Webページは自分で作れる」という感覚です。

HTML と CSS で作れるもの

- 自己紹介ページ
- 部活・クラス紹介ページ
- イベント告知ページ
- お店や作品の紹介ページ
- ポートフォリオの最初の1ページ

HTML と CSS だけでも、

「見る人に伝わる1枚のページ」 は十分作れます。

もくじ

1. Webページの仕組み — 開発環境の準備
2. ページの土台を作ろう — HTMLの基本構造
3. ヘッダーを作ろう — 見出し・テキスト・背景色
4. プロフィールカードを作ろう — 画像・横並び・影
5. 好きなもののリストを作ろう — リスト・装飾
6. リンクを作ろう — リンク・ボタン風デザイン
7. 発表会
8. 応用課題 — ダークモード & もっと
9. まとめ

Chapter 1

Webページの仕組み

ふだん見ているWebページの裏側

ブラウザで Web ページを開くと、裏側では **3種類のファイル形式** が読み込まれています。

ファイル形式	役割	やること
HTML	構造	ページに「何を置くか」を決める
CSS	見た目	「どう見えるか」を決める
JavaScript	動き	「どう動くか」を決める

この3つを書けば、Web ページが作れます。

例え話: 家を建てる

	役割	家で言うと
HTML	構造	柱・壁・窓の配置（間取り図）
CSS	見た目	壁紙の色・カーテン・家具（インテリア）
JavaScript	動き	自動ドア・照明のスイッチ（仕掛け）

HTMLだけだと **骨組みだけの家**。

CSSを足すと **きれいな家** になる。

JSを足すと **便利な家** になる。

もう少し具体的に

HTML（構造）

- ページのタイトル
- 写真
- 箇条書きのリスト
- リンク
- ボタン

→ 「何があるか」を定義する

→ 例えば: 自己紹介、作品紹介、お知らせ一覧

CSS（見た目）

- 背景の色
- 文字の大きさ
- 要素の配置（横並びなど）
- 角を丸くする
- 影をつける

→ 「どう見えるか」を決める

→ 例えば: かわいい、見やすい、アプリっぽい

JavaScript（動き）

- ボタンを押したら色が変わる
- クリックしたら数が増える
- 時間がたったら表示が変わる

→ 「何かが起きたら、何かをする」

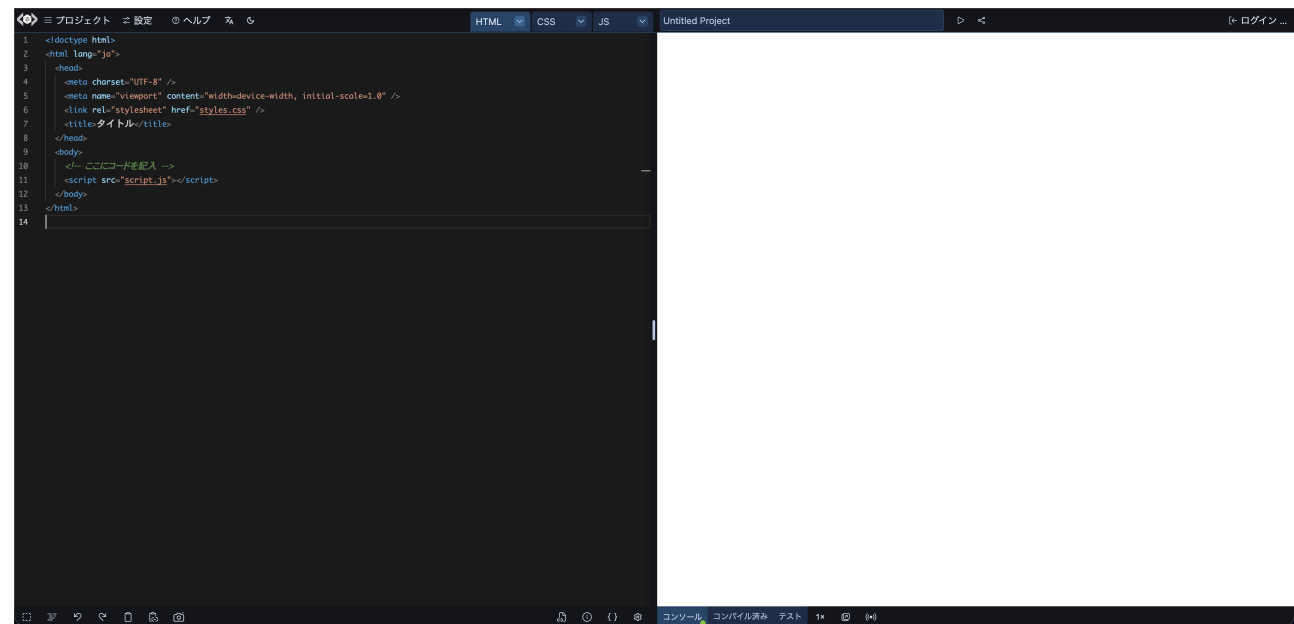
今日は HTML と CSS をメインに使い、時間があれば最後に少しでも JavaScript を体験します

開発環境: LiveCodes

今日は LiveCodes というWebサービスを使います。

- ブラウザだけで動く
- 左側: コードを書く場所
- 右側: プレビュー（結果がすぐ見える）

コードを書き換えると、自動で右側の表示が更新されます。



LiveCodes を開こう

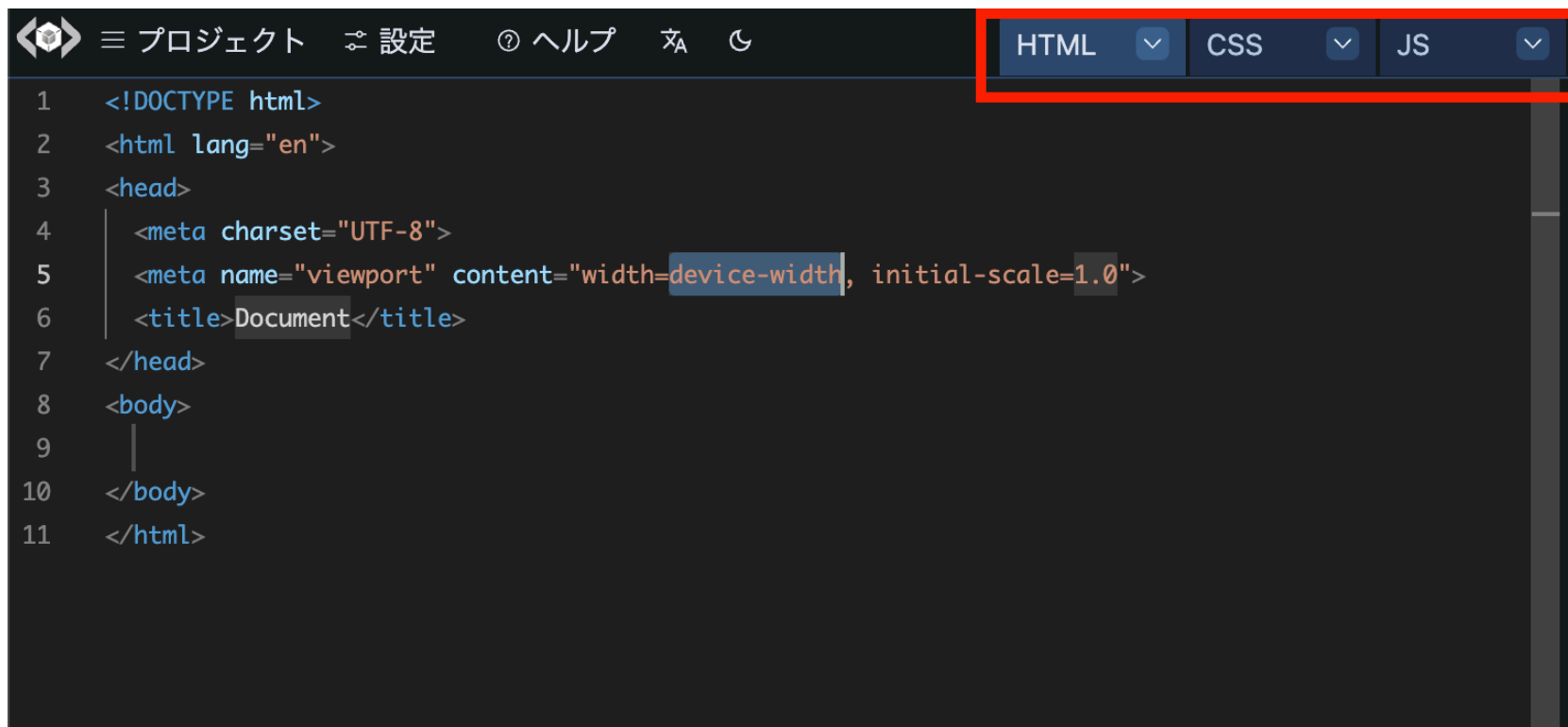
1. <https://v48.livecodes.io/?appLanguage=ja&formatOnsave=true&x=id/9ej3q5ctbwe> をクリック
2. 画面が表示されたら準備完了

確認すること

- 左側に編集画面が表示される
- 右側にプレビューが表示される
- コードを変えると **自動で画面が更新される**

ファイルの切り替え方

画面上部にある
「HTML」「CSS」を押
すと切り替えられま
す。



The screenshot shows a code editor interface with a dark theme. At the top, there is a navigation bar with icons for project, settings, help, and search. To the right of these icons is a language selection bar with three buttons: 'HTML', 'CSS', and 'JS'. Each button has a small downward arrow. A red rectangular box highlights this entire language selection bar. Below the bar, the editor displays HTML code with line numbers 1 through 11. The code is as follows:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Document</title>
7 </head>
8 <body>
9   |
10 </body>
11 </html>
```

Chapter 2

ページの土台を作ろう

HTMLの基本構造

HTML を開くと、こんなコードが入っています。

```
<!DOCTYPE html>
<html lang="ja">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="stylesheet" href="styles.css" />
    <title>自己紹介ページ</title>
  </head>
  <body>
    <!-- ヘッダーのHTML -->
    <div class="header">
      <h1>田中 はなこ</h1>
      <p>Web開発をはじめたばかりの大学生です！</p>
    </div>
    <!-- 省略 -->
  </body>
</html>
```

各パーツの意味

最初は **おまじない** だと思ってOKです。

大事なのは `<body>` と `</body>` の間に書く という事。

コード	意味
<code><!DOCTYPE html></code>	「これはHTMLです」という宣言
<code><head></code>	ページの設定
<code><link></code>	CSSファイルを読み込む
<code><body></code>	ページの中身 （ここに書いたものが表示される）

Chapter 3

ヘッダーを作ろう

このチャプターで作るもの

ピンクの背景に、
白い文字で **名前** と **一言メッセージ**
を表示するヘッダーを作ります。

できるようになること

- ページの最初に「何のページか」
を伝えられる
- 文字と背景色で第一印象を作れる

田中 はなこ

Web開発をはじめたばかりの大学生です！

コードにする前に、完成画面を部品で見よう

- まず大きなヘッダーを1つ作る
- その中に「名前」と「ひとこと」を入れる
- ヘッダー全体に背景色をつける



HTMLの「タグ」とは

HTMLは **タグ** という目印を使って書きます。

タグをつけることで、ブラウザに **「これは見出しだよ」「これは文章だよ」** と伝えます。

```
<h1>田中 はなこ</h1>
```

- `<h1>` → 「ここから見出し」（開始タグ）
- `</h1>` → 「見出しここまで」（終了タグ。 `/` がつく）

タグで囲まれた部分が、そのタグの種類に応じて表示される。

今回使うタグ

タグ	意味	表示
<code><h1></code>	見出し（一番大きい）	太く大きい文字
<code><p></code>	段落（パラグラフ）	ふつうの文章
<code><div></code>	グループ化する箱	見た目は何もない（CSSで変える）

`<div>` は **他のタグをまとめるための入れ物** として使います。

なぜまとめるの？ → まとめることで **CSSをグループ単位で適用** できる。

例: ヘッダー全体に背景色をつけたい → `<h1>` と `<p>` を `<div>` でまとめて、その `<div>` に背景色を指定する。

ヘッダーのHTML

HTML の `<body>` の下に次のような記述があります。

```
<!-- ヘッダーのHTML -->  
<div class="header">  
  <h1>田中 はなこ</h1>  
  <p>Web開発をはじめたばかりの大学生です！</p>  
</div>
```

名前や説明文を変更して画面が更新されることを確認しましょう。

CSSの書き方

次に見た目を整えます。

```
セクタ {  
  プロパティ: 値;  
}
```

```
body {  
  color: pink;  
}
```

← 「body に対して」
← 「文字色を pink にする」

- セクタ = どの要素に適用するか
- プロパティ = 何を変えるか
- 値 = どう変えるか

class属性 — HTMLとCSSを紐づける

```
<div class="header">
```

↑ この `class="header"` が **名札** の役割。

```
.header {  
  background-color: pink;  
}
```

↑ CSSでは `.header` と書くと、`class="header"` の要素に適用される。

class は名札、CSSはその名札を見てスタイルを適用する。

ヘッダーのCSS

CSS に追加：

```
/* ヘッダーのCSS */  
.header {  
  background-color: pink;          /* 背景色（ピンク） */  
  text-align: center;              /* 文字を中央揃え */  
  padding: 40px 20px;              /* 内側の余白（上下40px、左右20px） */  
  border-radius: 0 0 20px 20px;    /* 角を丸くする（左上・右上・右下・左下） */  
}
```

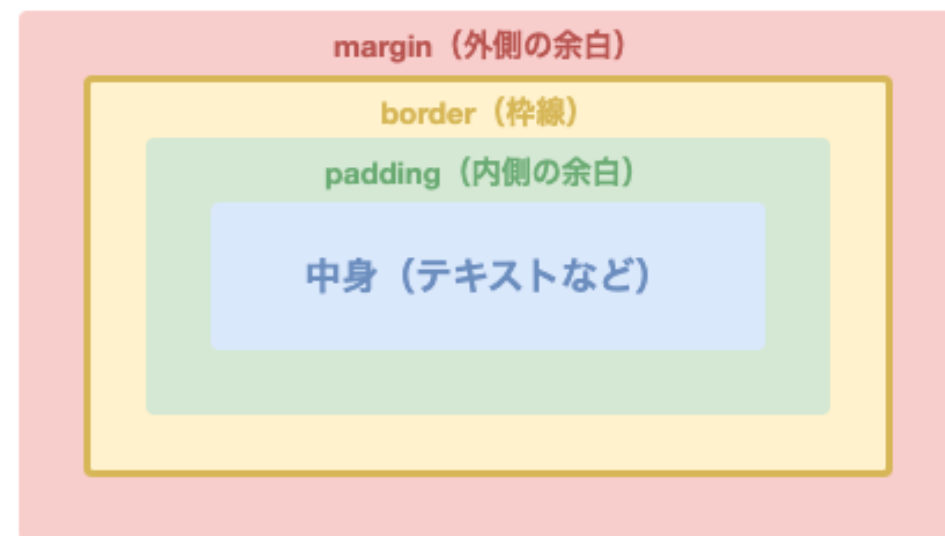
`px` = ピクセル。画面上の点の数で大きさを指定する単位。現実でいうと「cm」や「m」のようなもの。

padding と margin

bodyの余白をなくすために `margin: 0;`
`padding: 0;` と書きましたが、**余白には2種類**
あります

- **padding** = 内側の余白
- **margin** = 外側の余白

デフォルトでブラウザは `body` に余白をつけていますが、今回は自分で余白をコントロールしたいので、両方とも0にしています。



padding: 内側の余白

要素の**内側**の余白が padding
内側に余白を作ることによってテキストが
枠にくっつかず、見やすくなります。

✗ padding なし

テキストが枠に
くっついてしまう…

padding: 0

✓ padding あり

テキストに
ゆとりがある！

padding: 24px

margin: 外側の余白

要素の**外側**の余白が margin
外側に余白を作ること、要素同士
がくっつかず、見やすくなります。

✖ margin なし

カード1

カード2

カード3

margin-top: 0

✔ margin あり

カード1

カード2

カード3

margin-top: 20px

padding / margin の値の書き方

```
/* 値が1つ → 上下左右すべて同じ */  
padding: 24px;
```

```
/* 値が2つ → 上下 / 左右 */  
padding: 40px 20px;  
/*           ↑上下  ↑左右 */
```

```
/* 値が4つ → 上 / 右 / 下 / 左 */  
padding: 10px 15px 20px 25px;  
/*           ↑上  ↑右  ↑下  ↑左 */
```

あとで、padding と margin を使う場面がたくさん出てくるので、違いを意識してみてください。

ヘッダーの文字のCSS

```
/* ヘッダーの文字のCSS */  
.header h1 {  
  margin: 0;  
  color: white;  
}  
  
.header p {  
  margin: 8px 0 0 0; /* 上 / 右 / 下 / 左 */  
  color: white;  
}
```

`.header h1` = 「header の中にある h1」という意味。

これで **ヘッダーの中の見出しだけ** にスタイルを当てられる。

ここまでの結果

ブラウザのプレビューを確認してみましょう。

今できるようになったこと

- ページのタイトル部分を自分で作れる
- 背景色や文字色で雰囲気を変えられる

うまく表示されない場合は、タグの閉じ忘れがないか確認してください。

田中 はなこ

Web開発をはじめたばかりの大学生です！

Chapter 4

プロフィールカードを作ろう

このチャプターで作るもの

丸い写真と自己紹介テキストを **横並び** で表示するカードを作ります。

できるようになること

- 「ひとまとまりの情報」をカードとして見やすく置ける
- 写真と文章をセットで紹介できる



こんにちは！田中はなこです。

東京に住んでいて、カフェ巡りと猫が好きです。最近プログラミングを始めました！

コードにする前に、完成画面を部品で見よう

- まず大きなカードを1つ作る
- その中に「写真」と「説明文」を入れる
- 説明文の中に文章を2つ入れる



新しいタグ: `<img>`

画像を表示するタグ。終了タグがない 特殊なタグです。

```

```

属性	意味
<code>src</code>	画像ファイルのURL（どの画像を表示するか）
<code>alt</code>	画像が読み込めなかった時に表示される代替テキスト

プロフィールのHTML

`<!-- プロフィールのHTML -->` と書かれた部分の下側にコードを4行追加しましょう。

```
<div class="container">
  <div class="card">
    <div class="profile">
      
      <!-- プロフィールのHTML -->
      <div class="profile-text">
        <p>こんにちは！田中はなこです。</p>
        <p>東京に住んでいて、カフェ巡りと猫が好きです。最近プログラミングを始めました！</p>
      </div>
    </div>
  </div>
</div>
```

カードのCSS

```
/* カードのCSS */  
.card {  
  background-color: white;      /* 白い背景 */  
  border-radius: 16px;          /* 角を丸く */  
  padding: 24px;                /* 内側の余白 */  
  margin-top: 20px;             /* カード同士の間隔 */  
  box-shadow: 0 2px 12px rgba(0, 0, 0, 0.08); /* 影 */  
}
```

横並びにする: Flexbox

写真とテキストを 横に並べたい 時は

`display: flex` を使います。

```
/* 横並びにする: Flexbox */  
.profile {  
  display: flex;  
  align-items: center;  
  gap: 20px;  
}
```

`display: flex` → 子要素が横に並ぶ

`align-items: center` → 縦方向を真ん中に

`gap: 20px` → 要素同士の間隔

✗ flexなし (デフォルト)

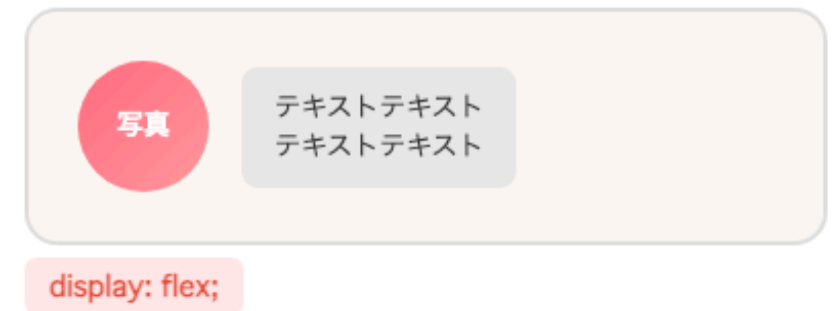
要素が縦に積み重なる



↓ **display: flex** を追加 ↓

✓ flexあり

要素が横に並ぶ！



写真を丸くする

```
/* 写真を丸くする */  
.profile-img {  
  width: 100px;  
  height: 100px;  
  border-radius: 50%;          /* 50%で正円 */  
  object-fit: cover;          /* 枠に合わせてトリミング */  
  border: 3px solid pink;     /* ピンクの枠線 */  
}
```

`border-radius: 50%` で正方形が **正円** になる。よく使うテクニック。

テキストのCSS

```
/* テキストのCSS */  
.profile-text p {  
  margin: 4px 0;  
  font-size: 15px;  
  line-height: 1.6;  
}
```

`line-height: 1.6` = 行間を1.6倍にする。
文章が読みやすくなります。

カード全体の幅を制限する

今のままだと画面全体にカードが広がってしまうので、CSSに追加しましょう。

```
/* カード全体の幅を制限する */  
.container {  
  max-width: 600px;      /* 横幅を制限 */  
  margin: 0 auto;        /* 中央寄せ */  
  padding: 20px;         /* 内側の余白 */  
}
```

ここまでの結果

ヘッダー + プロフィールカードが表示されていればOK！

今できるようになったこと

- プロフィールや商品紹介のような情報ブロックを作れる
- `display: flex` で画像と文章を横並びにできる

田中 はなこ

Web開発をはじめたばかりの大学生です！



こんにちは！田中はなこです。
東京に住んでいて、カフェ巡りと猫が好きです。最近プログラミングを始めました！

メモ: 作業を一時保存したい時は

LiveCodes は **ブラウザのタブを閉じる / 別のPCで開く** と、書いたコードがそのままでは引き継がれないことがあります。

時間がかかりそう・別の日に続きをやりたい場合は、ここまでの作業を **ZIPファイル** として手元に保存しておくで安心です。

- 自分のPCにコードが **ファイルとして残る**
- あとから **同じ続きから再開** できる
- もし操作ミスでコードが消えても **戻せる**

このスライドは「保存しておきたい時に見返す」用のメモです。
続けて進める場合は次の Chapter 5 へ進んでOK。

メモ: ZIP として保存する(エクスポート)

1. 画面の **プロジェクト** メニューを開く
2. **エクスポート** にマウスを合わせる
3. **ソースをエクスポート (ZIP)** をクリック
4. ZIPファイルがダウンロードされる

ダウンロードフォルダに ZIP ファイルが保存されます。

名前を `自己紹介ページ.zip` のように分かりやすく変えておくと安心です。



メモ: 保存した ZIP から再開する(インポート)

次に LiveCodes を開いた時に ZIP ファイルを読み込むと続きから始められます。

1. LiveCodes を開く:

[https://v48.livecodes.io/?
appLanguage=ja&formatOnsave=true&x=id/9ej3q5ctbwe](https://v48.livecodes.io/?appLanguage=ja&formatOnsave=true&x=id/9ej3q5ctbwe)

2. **プロジェクト メニュー → インポート**
をクリック

3. **ローカルファイルをインポート** をクリックして、保存した ZIP を選ぶ



Chapter 5

好きなもののリストを作ろう

このチャプターで作るもの

好きなものを 箇条書き で表示するリストを作ります。

できるようになること

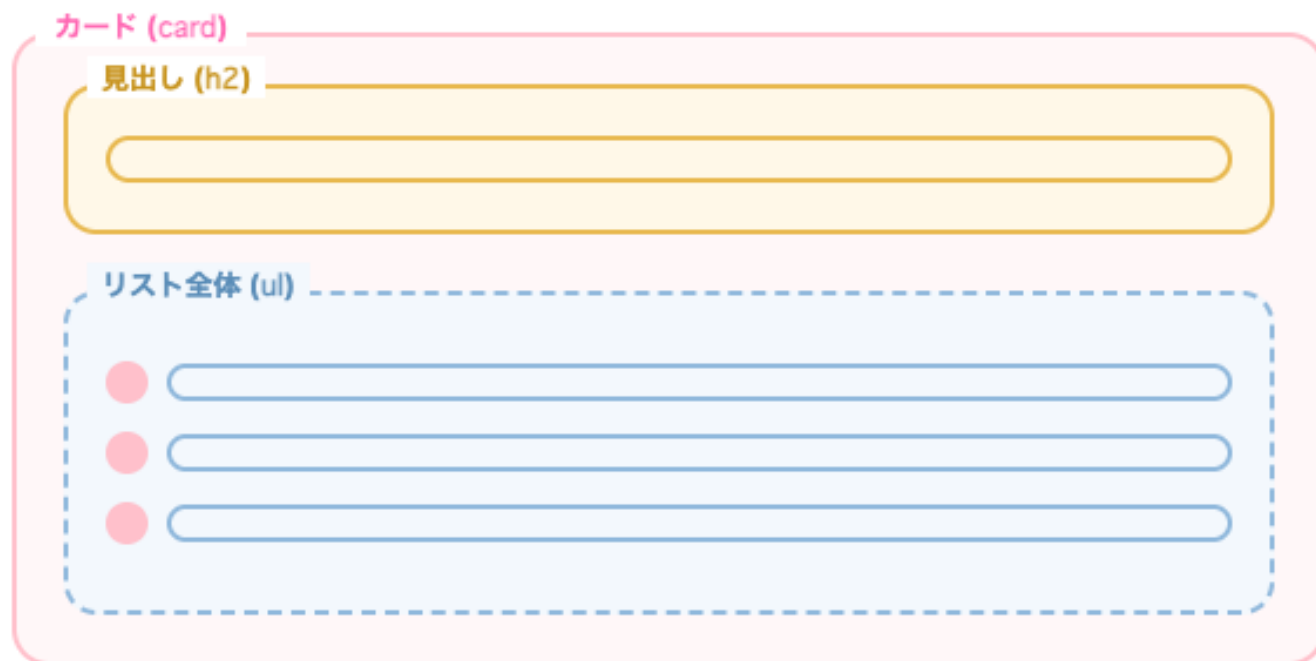
- 情報を箇条書きで整理して見せられる

好きなもの

- カフェ巡り
- 猫
- ラーメン

コードにする前に、完成画面を部品で見よう

- カードの中に見出しを1つ置く
- その下にリスト全体を置く
- リストの中に項目を1つずつ入れる



新しいタグ: `<h2>`, `<ul>`, `<li>`

タグ	意味	表示
<code><h2></code>	小見出し（h1 の次に大きい）	太い中サイズ文字
<code></code>	順番なしリスト（箇条書き）	・が付いたリスト
<code></code>	リストの各項目	1つ1つの項目

```
<ul>          ← リストの開始
  <li>項目1</li>
  <li>項目2</li>
  <li>項目3</li>
</ul>         ← リストの終了
```

リストのHTML

HTML で、`<!-- === 好きなもののリスト === -->` と書かれた場所を次のように穴埋めしましょう。

```
<!-- === 好きなもののリスト === -->
<div class="card">
  <h2>好きなもの</h2>
  <!-- リストのHTML -->
  <ul>
    <li>カフェ巡り</li>
    <li>猫</li>
    <li>ラーメン</li>
  </ul>
</div>
```

見出しのCSS

```
/* 見出しのCSS */  
.card h2 {  
  font-size: 20px;  
  margin: 0 0 12px;  
  color: hotpink;  
}
```

`.card h2` = 「card 中にある h2」にだけ適用。

すべての h2 ではなく、**カード内**の見出しだけ をピンクにします。

ここまでの結果

ヘッダー + プロフィール + 好きなもののリスト
が表示されていればOK！

今できるようになったこと

- 好きなもの、特徴、ルールなどを見やすく並べられる
- HTMLを増やさなくても CSS で飾りを足せる

田中 はなこ

Web開発をはじめたばかりの大学生です！



こんにちは！田中はなこです。

東京に住んでいて、カフェ巡りと猫が好きです。最近プログラミングを始めました！

好きなもの

- カフェ巡り
- 猫
- ラーメン

Chapter 6

リンクを作ろう

このチャプターで作るもの

ボタン風の **リンク** を横並びで表示します。

できるようになること

- 自分のSNSや作品ページへ案内できる
- リンクを「押したくなる見た目」にできる

リンク

jig.jp

小山高専 HP

コードにする前に、完成画面を部品で見よう

- カードの中に見出しを置く
- その下にリンクを並べる箱を作る
- 1つ1つのリンクをボタン風に見せる



リンクのHTML

- `<!-- リンクのHTML -->` の下にリンクを追加しましょう。

```
<!-- リンク -->
<div class="card">
  <h2>リンク</h2>
  <div class="links">
    <a target="_blank" href="https://jig.jp" class="link">jig.jp</a>
    <!-- リンクのHTML -->
    <a target="_blank" href="https://www.oyama-ct.ac.jp/" class="link">小山高専</a>
  </div>
</div>
```


`<a>` タグ = リンクを作るタグ

属性	意味
<code>target</code>	<code>"_blank"</code> を指定するとブラウザの新しいタブで開きます。
<code>href</code>	リンク先のURL（例: <code>https://jig.jp</code> ）

ボタン風のデザイン

```
/* リンクのCSS */  
.links {  
  display: flex;  
  gap: 10px;  
  margin-top: 8px;  
}  
  
.link {  
  display: inline-block;  
  padding: 10px 20px;  
  background-color: pink;  
  color: white;  
  text-decoration: none;  
  border-radius: 25px;  
  font-size: 14px;  
  font-weight: bold;  
}
```

`text-decoration: none` → リンクの下線を消す

ブラウザはデフォルトで `<a>` タグに青い文字 + 下線をつけます。

`text-decoration: none` と `color: white` で、それをリセットしてボタン風に見せています。

ホバー効果をつける

```
/* ホバー効果をつける */  
.link:hover {  
  background-color: hotpink;  
}
```

:hover とは？

マウ斯卡ーソルを **要素の上に乗せた時** に適用されるスタイル。

→ ボタンの上にマウスを乗せると、色が少し濃いピンクに変わる！

基礎パート完成！

ここまでで自己紹介ページの土台ができました 🎉

ここまででできるようになったこと

- 1枚の自己紹介ページを最初から作れる
- 色、余白、角丸、影で見た目を整えられる
- リスト、タグ、リンクで情報を整理して伝えられる

田中 はなこ

Web開発をはじめたばかりの大学生です！



こんにちは！田中はなこです。
東京に住んでいて、カフェ巡りと猫が好きです。最近プログラミングを始めました！

好きなもの

- カフェ巡り
- 猫
- ラーメン

リンク

jig.jp

小山高専 HP

今日使ったHTMLタグ

タグ	意味
<code><h1></code>	見出し
<code><h2></code>	小見出し
<code><p></code>	段落
<code><div></code>	グループ化する箱

タグ	意味
<code></code>	画像
<code></code>	箇条書き
<code></code>	リストの各項目
<code><a></code>	リンク

今日使ったCSSプロパティ

プロパティ	意味
background-color	背景色
color	文字色
font-size	文字の大きさ
font-weight	文字の太さ
line-height	行間
text-decoration	テキストの装飾
padding	内側の余白
margin	外側の余白
width / height	要素の幅と高さ

プロパティ	意味
max-width	要素の最大幅
border	枠線
border-radius	角を丸くする
box-shadow	影
display	要素の表示方法
align-items	Flexboxで縦方向の配置
gap	要素同士の間隔
flex-wrap	Flexboxで折り返しを指定
:hover	ホバー時のスタイル

Chapter 7

発表会

発表会

今回作成したWebサイトを使用して自己紹介をしてみましょう！

発表の内容

- 自分のページを画面に映す
- ページを使って **自己紹介**
- 応用課題をやった人はそれも見せてください
- 「どこを工夫したか」を1つ話せるとさらによいです

例えば...

「ヘッダーの背景色を青色にして、かっこいい雰囲気になりました！」

「プロフィールカードの写真を四角にして、クールな印象になりました！」

「カードを1つ追加して、好きな本を紹介しました！」

Chapter 8

応用課題

目次 (1/2)

- **ダークモード切替** (★)
 - 配色を明るい色  暗い色に切り替え
- **いいねボタン** (★)
 - 押すたびに数字が増えるボタン
- **ページ訪問回数** (★)
 - ページが訪問された回数を表示、ブラウザを閉じてでも残る
- **現在時刻の表示** (★)
 - ヘッダーに現在時刻を記載
- **タブ切り替え** (★★)
 - 好きなもの / 苦手なものをタブで切替
- **画像スライドショー** (★★)
 - ← / → で画像を切り替え、ループする

目次 (2/2)

- **カスタムカーソル** (★★)
 - カーソルが丸に変化 + 軌跡パーティクル
- **モーダル** (★★★)
 - 写真をクリックで拡大表示
- **ドラッグで並び替え** (★★★)
 - リスト項目をドラッグ&ドロップで順番変更
- **ミニクイズゲーム** (★★★)
 - 3 択クイズ、スコア表示付き
- **デザインのモダン化** (上級)
 - ページ全体をモダンなデザインに

応用課題の進め方

1. **ダークモード切替** — ここで JavaScript を体験します
2. 好きな応用課題を選んで自分のペースで進める
3. わからないことは **メンターに聞いてOK**

ダークモード切替

JavaScript を使います

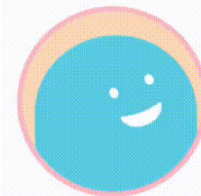
ダークモードとは

ページの配色を **明るい** ↔ **暗い** に切り替える機能。

なぜダークモードが使われている？

- **目に優しい** — 暗い場所で画面が眩しくない
- **バッテリー節約** — 有機ELディスプレイでは黒が省電力
- **ユーザーの好みに対応** — 多くのアプリ・サイトが標準搭載

YouTube、X、LINE、GitHub... ほぼすべての有名サービスが対応しています。



こんにちは！田中はなこです。

東京に住んでいて、カフェ巡りと猫が好きです。最近プログラミングを始めました！

好きなもの

- カフェ巡り
- 猫
- ラーメン
- 音楽を聴くこと
- 写真を撮ること

できること・学んでいること

HTML

CSS

JavaScript

写真

デザイン

SNS

X

Instagram

GitHub

🌙 ダークモード

ダークモードの作り方

流れ

1. CSS — 暗い配色のスタイルを追加する
2. HTML — 切り替えボタンを追加する
3. JS — ボタンを押したらスタイルを切り替える

その前に: CSSでの色の指定方法

これまでは `pink` や `white` のような **色の名前** で色を指定してきました。

ただし色名は約140色しかなく、**ダークモード用の暗い色** は名前で表現できません。

→ より自由に色を指定するために **カラーコード** を使います。

カラーコード（# + 6桁の英数字）

```
background-color: #1a1a2e;  
color: #ffffff;
```

の後に 赤(R)・緑(G)・青(B) を2桁ずつ指定する。
色名より細かい色が指定できる。

カラーコードの読み方

6桁は **赤・緑・青** の順で、数字が大きいほどその色が強い。

- `#ff0000` → 赤が最大、他が0 → **赤**！
- `#0000ff` → 青が最大 → **青**！
- `#000000` → 全部0 → **黒**
- `#ffffff` → 全部最大 → **白**

カラーコードは覚えなくてOK。

ネットで「**カラーピッカー**」と検索すると好きな色を選べます。

Step 1: ダークモード用の CSS を追加

```
body.dark {  
  background-color: #1a1a2e;  
  color: #e0e0e0;  
}  
  
body.dark .header {  
  background-color: #2d2d5e;  
}  
  
body.dark .card {  
  background-color: #16213e;  
  box-shadow: 0 2px 12px rgba(0, 0, 0, 0.3);  
}  
  
body.dark .card h2 {  
  color: #ffa0c4;  
}  
  
body.dark .link {  
  background-color: #2d2d5e;  
  color: #e0e0e0;  
}  
  
body.dark .link:hover {  
  background-color: #3d3d7e;  
}
```

body.dark って何？

```
body.dark { ... }
```

これは「body に `dark` というクラスが **ついている時だけ** 適用する」という意味。

```
<body>                → body.dark のスタイルは適用されない  
<body class="dark"> → body.dark のスタイルが適用される
```

JavaScript で **このクラスをつけたり外したり** することで、切り替えを実現します。

Step 2: 切り替えボタンのHTML

HTML の container の中、一番下に追加：

```
<button class="dark-mode-btn" id="darkModeBtn">  
  🌙 ダークモード  
</button>
```

新しい要素

要素	意味
<code><button></code>	クリックできるボタンを作るタグ
<code>id="darkModeBtn"</code>	この要素に ユニーク（唯一の名前） をつける

class と id の違い

	class	id
書き方	class="card"	id="darkModeBtn"
CSS	.card { }	#darkModeBtn { }
使える数	同じ名前を 何個でも 使える	1ページに 1つだけ
用途	スタイル用（同じ見た目のものに）	JS で特定の要素を見つける用

ボタンのCSS

```
.dark-mode-btn {  
  display: block;  
  margin: 24px auto 40px;  
  padding: 10px 24px;  
  border: 2px solid #ccc;  
  border-radius: 25px;  
  background: none;  
  font-size: 14px;  
  cursor: pointer;  
  color: inherit;  
  transition: border-color 0.3s;  
}  
  
.dark-mode-btn:hover {  
  border-color: #ff7eb3;  
}
```


Step 3: JavaScriptで切り替える

script.js に追加：

```
const darkModeBtn = document.getElementById("darkModeBtn");

darkModeBtn.addEventListener("click", function() {
  document.body.classList.toggle("dark");

  if (document.body.classList.contains("dark")) {
    darkModeBtn.textContent = "☀️ ライトモード";
  } else {
    darkModeBtn.textContent = "🌙 ダークモード";
  }
});
```

JavaScript を1行ずつ解説 (1/3)

```
const darkModeBtn = document.getElementById("darkModeBtn");
```

パーツ	意味
<code>const darkModeBtn</code>	<code>darkModeBtn</code> という名前の 変数 （データを入れる箱）を作る
<code>document.getElementById(...)</code>	ページの中から id が指定された要素を 探す
<code>"darkModeBtn"</code>	id が <code>darkModeBtn</code> の要素を指定

→ 「ページから `darkModeBtn` ボタンを見つけて、`darkModeBtn` に入れておく」

JavaScript を1行ずつ解説 (2/3)

```
darkModeBtn.addEventListener("click", function() {  
    // ここに「クリックされた時にやること」を書く  
});
```

パーツ	意味
<code>addEventListener</code>	「〇〇が起きたら△△する」を登録する
<code>"click"</code>	「クリック」というイベント
<code>function() { ... }</code>	クリックされた時に実行する処理

→ 「`darkModeBtn` がクリックされたら、この処理を実行する」

JavaScript を1行ずつ解説 (3/3)

```
document.body.classList.toggle("dark");
```

パーツ	意味
<code>document.body</code>	<code><body></code> 要素
<code>.classList</code>	その要素が持つクラスの一覧
<code>.toggle("dark")</code>	<code>dark</code> クラスがなければ つける 、あれば 外す

→ 「body の dark クラスを ON/OFF する」

動作確認

ボタンをクリックしてみましょう！

- 1回押す → 背景が暗くなる（ダークモード ON）
- もう1回押す → 元に戻る（ダークモード OFF）

その他の応用課題（自走用）

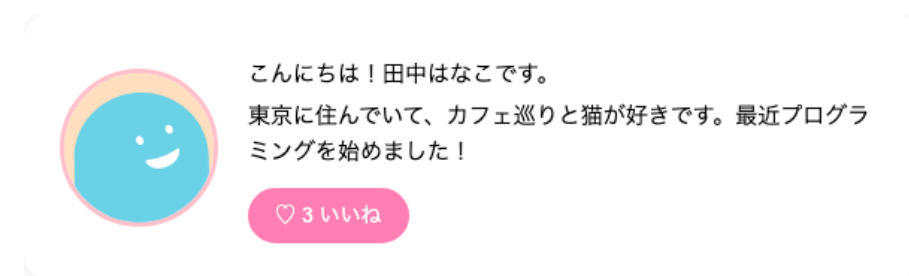
ダークモードが終わった人は好きなものに挑戦！
手順とヒントを読みながら自分のペースで進めてください。
答えは各課題の下のトグルを開くと見られます。

★ いいねボタン

完成イメージ: プロフィールカードに ♡ ボタン。押すたびに数字が 1 ずつ増える。

手順

1. HTML: プロフィールカード内に `<button id="likeBtn">♡ 0 いいね</button>` を追加
2. CSS: ボタンにピンクの枠線・角丸のスタイルを作る
3. JS: `let likeCount = 0` で数を記録。クリックで +1 して表示を更新



ヒント

- ダークモードで学んだ `getElementById` + `addEventListener` がそのまま使える
- `likeBtn.textContent = "♡ " + likeCount + " いいね"` で表示を書き換え

★ いいねボタン — 答え

解答例: `examples/advanced/b-like-button.html`

★ ページ訪問回数

完成イメージ: ヘッダーに「このページは N 回目の訪問です」と表示。ブラウザを閉じてても回数が残る。

手順

1. HTML: ヘッダー内に `<div id="visitCount">`
`</div>` を追加
2. JS: `localStorage.getItem("visitCount")` で
前回の値を取得
3. +1 して `localStorage.setItem()` で保存し、
画面に表示

田中 はなこ

Web開発をはじめたばかりの大学生です！

このページは 1 回目の訪問です

ヒント

- `localStorage` はブラウザにデータを保存する仕組み（閉じてても消えない）
- 初回は値がないので `null` が返る → `0` からスタート
- `Number()` で文字列を数値に変換する

★ ページ訪問回数 — 答え

解答例: `examples/advanced/c-visit-counter.html`

★ 現在時刻の表示

完成イメージ: ヘッダーに「🕒 14:32:05」のように現在時刻が表示され、毎秒更新される。

手順

1. HTML: ヘッダー内に `<div id="clock">`
`</div>` を追加
2. JS: `new Date()` で現在時刻を取得する関数を作る
3. JS: `setInterval(関数, 1000)` で1秒ごとに関数を呼び出す

田中 はなこ

Web開発をはじめたばかりの大学生です！

🕒 14:55:05

ヒント

- `now.getHours()` , `now.getMinutes()` , `now.getSeconds()` で時・分・秒が取れる
- `String(9).padStart(2, "0")` → `"09"` のように0埋めできる

★ 現在時刻の表示 — 答え

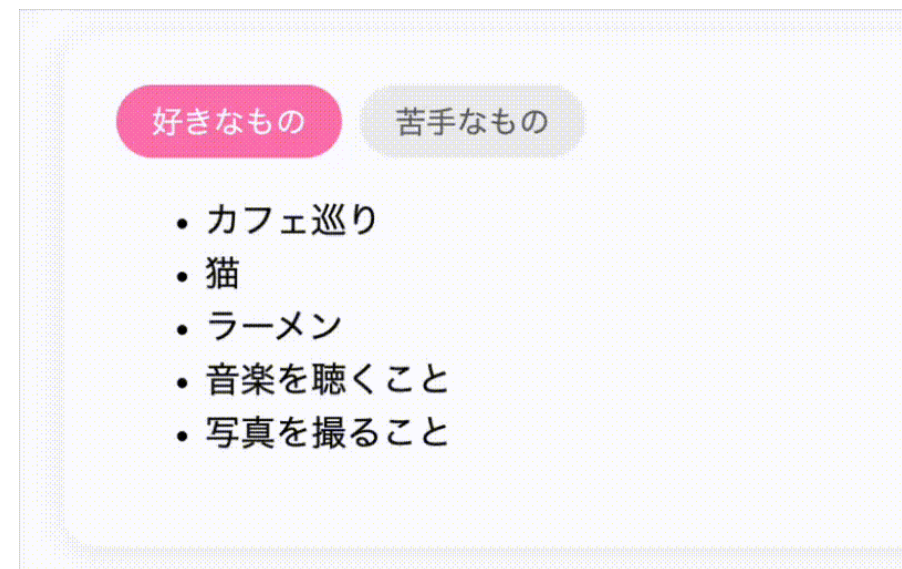
解答例: <examples/advanced/d-clock.html>

★★ タブ切り替え

完成イメージ: 「好きなもの」カードに2つのタブ。
クリックで「好きなもの」と「苦手なもの」の表示
が切り替わる。

手順

1. HTML: タブボタン2つ + 中身の div を2つ用意
(片方は非表示)
2. CSS: `.tab-content { display: none }` +
`.tab-content.active { display: block }`
3. JS: タブクリックで全タブを非アクティブにして
から、押されたタブだけアクティブに



ヒント

- ボタンに `data-tab="likes"` のようにデータ属性をつけると、どのタブが押されたか判定できる
- `this.getAttribute("data-tab")` でクリックされたタブの値を取得
- `#tab-likes` , `#tab-dislikes` のように id で中身を切り替える

★★ タブ切り替え — 答え

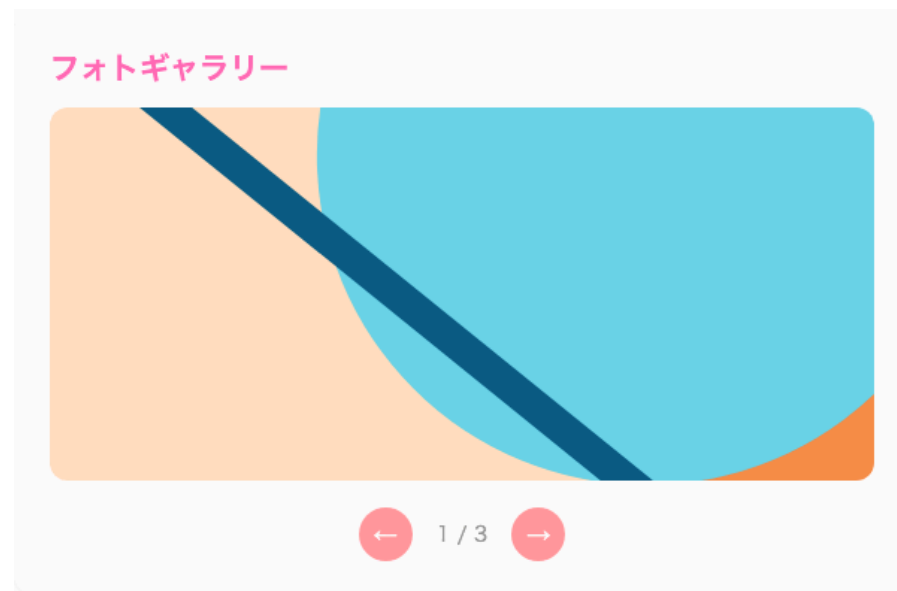
解答例: `examples/advanced/e-tabs.html`

★★ 画像スライドショー

完成イメージ: フォトギャラリーに画像1枚 + ← → ボタン。ボタンで画像が切り替わり「1 / 3」と表示。最後の次は最初に戻る。

手順

1. HTML: 新しいカードに `<img id="slideshowImg">` + 前へ/次へボタン + カウンター
2. JS: 画像URLを配列で管理。 `let currentPhoto = 0` で今何枚目か記録
3. JS: ボタンクリックでインデックスを変更し、画像の `src` とカウンターを更新



ヒント

- 配列: `const photos = ["url1", "url2", "url3"]`
- `photos[currentPhoto]` で現在の画像URLを取得
- 最後 → 最初に戻す: `currentPhoto = (currentPhoto + 1) % photos.length`

★★ 画像スライドショー — 答え

解答例: <examples/advanced/f-slideshow.html>

★★ カスタムカーソル

完成イメージ: マウスカーソルがピンクの丸に変わる。動かすとキラキラしたパーティクルが軌跡に残って消える。

手順

1. HTML: `<div class="custom-cursor" id="customCursor"></div>` を追加
2. CSS: 丸いスタイル + `position: fixed` + `pointer-events: none`
3. JS: `mousemove` イベントでカーソル位置を更新。一定間隔でパーティクルを生成 → 自動削除

カスタムカーソル

マウスを動かすとピンクの丸がついてくる
軌跡にパーティクルが残って消える ✨

ヒント

- `e.clientX`, `e.clientY` でマウス位置が取れる
- パーティクルは `document.createElement("div")` で動的に作り
`document.body.appendChild()` で追加
- `setTimeout(function() { particle.remove(); }, 600)` で自動削除
- `@keyframes` でフェードアウトアニメーションを定義

★★ カスタムカーソル — 答え

解答例: <examples/advanced/g-cursor.html>

★★★ モーダル（ポップアップ）

完成イメージ: プロフィール写真をクリック → 画面全体が暗くなり写真が拡大表示。暗い部分 or × ボタンで閉じる。

手順

1. HTML: オーバーレイ div（暗い背景 + 拡大画像 + 閉じるボタン）を追加
2. CSS: `position: fixed` で画面全体を覆い、`display: none` で初期非表示
3. JS: 写真クリック → `.show` 追加。×クリック or 背景クリック → `.show` 削除

田中 はなこ

Web開発をはじめたばかりの大学生です！



こんにちは！田中はなこです。
東京に住んでいて、カフェ巡りと猫が好きです。最近プログラミングを始めました！

好きなもの

ヒント

- `position: fixed; top: 0; left: 0; width: 100%; height: 100%` で全画面
- 背景は `rgba(0, 0, 0, 0.7)` で半透明の黒
- 背景自体のクリック判定: `if (e.target === modal)` で区別

★★★ モーダル — 答え

解答例: <examples/advanced/h-modal.html>

★★★ ドラッグで並び替え

完成イメージ: 好きなもののリストの項目をドラッグ&ドロップで順番変更。ドラッグ中は半透明、ドロップ先はハイライト。

手順

1. HTML: `<ul>` に `id="likesList"` を追加し、
各 `<li>` に `draggable="true"` を追加
2. CSS: `.dragging` (半透明) `.drag-over` (ハイライト) を定義
3. JS: `dragstart` / `dragend` / `dragover` / `drop`
の4イベントを処理

好きなもの

- カフェ巡り
- 猫
- ラーメン
- 音楽を聴くこと
- 写真を撮ること

ヒント

- `e.preventDefault()` を `dragover` と `drop` で呼ぶ（必須）
- `insertBefore()` で DOM 上の順番を入れ替える
- ドラッグ元とドロップ先のインデックスを比較して前後を判定

★★★ ドラッグで並び替え — 答え

解答例: <examples/advanced/i-drag.html>

★★★ ミニクイズゲーム

完成イメージ: 「私の好きな食べ物は？」などの3択クイズ。正解は緑、不正解は赤。全3問、スコア表示付き。

手順

1. HTML: クイズ用カード（質問・選択肢エリア・結果・次へボタン・スコア）を追加
2. JS: クイズデータを配列で定義 `[{ question, options, answer }, ...]`
3. JS: `showQuiz()` でボタンを動的生成。クリックで正解判定→スタイル変更→スコア更新

ミニクイズ

Q1. 私の好きな食べ物は？

ラーメン

寿司

カレー

○ 正解！

次の問題 →

ヒント

- `document.createElement("button")` でボタンを動的に作れる
- `setAttribute("data-index", i)` でどの選択肢かを記録
- 二重クリック防止に `let quizAnswered = false` フラグを使う

★★★ ミニクイズゲーム — 答え

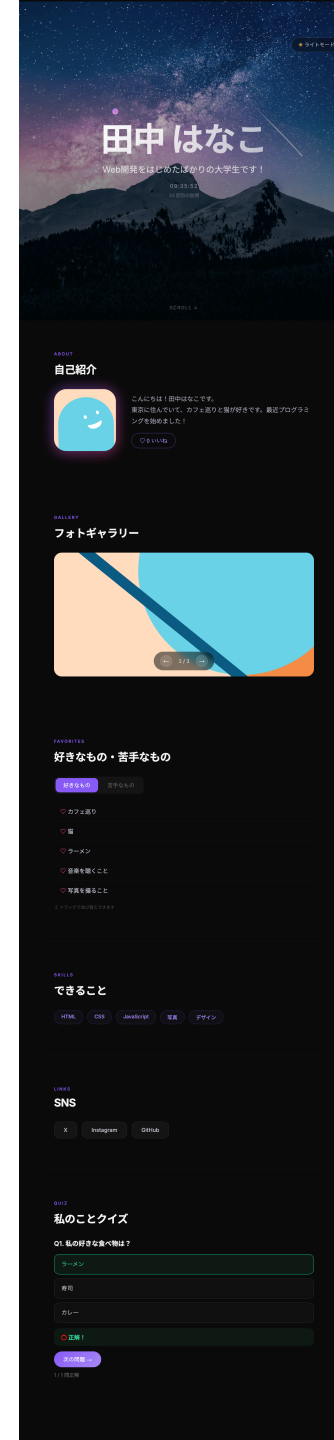
解答例: <examples/advanced/j-quiz.html>

デザインのモダン化（上級チャレンジ）

完成イメージ: これまで作ったページを、モダンなページに変身させる。

やること

1. ヒーローセクション — `height: 100vh` + 背景画像 + グラデーションオーバーレイ
2. ダークテーマ — 背景 `#0a0a0a`、文字 `#f5f5f5`
3. Google Fonts — `Inter` フォントの読み込み
4. カード廃止 → セクション + 区切り線レイアウト
5. スクロールアニメーション — `IntersectionObserver` + `CSS transition`



ヒント

- `background: url(...) center/cover no-repeat` で背景画像
- `linear-gradient` を重ねて画像の上にオーバーレイ
- `transform: translateX(-60px)` → `translateX(0)` で横からスライドイン

デザインのモダン化 — 答え

解答例: <examples/demo-modern.html>

Chapter 9

まとめ

今日やったこと

できるようになったこと

- 名前、写真、好きなものを載せたページを作れた
- CSSで色や余白や配置を調整できた
- リンクやボタン風デザインでページを使いやすくなった

使った主な道具

- HTML: `h1` `h2` `p` `div` `img` `ul` `li` `a`
- CSS: 背景色・文字色・配置・角丸・影・Flexbox・ホバー効果

田中 はなこ

Web開発をはじめたばかりの大学生です！



こんにちは！田中はなこです。
東京に住んでいて、カフェ巡りと猫が好きです。最近プログラミングを始めました！

好きなもの

- カフェ巡り
- 猫
- ラーメン

リンク

jig.jp

小山高専 HP

次に作れるもの

作れるもの	使う知識
部活紹介ページ	見出し・画像・リスト・リンク
作品紹介ページ	カード・写真・説明文・ボタン
イベント告知ページ	タイトル・日時・場所・申込みリンク
お店紹介ページ	写真・メニュー・地図リンク
ポートフォリオ	自己紹介・スキル・SNS・作品リンク

次に学ぶとよいこと

トピック	できること
レスポンシブ	スマホ対応のデザインが作れる
JavaScript	もっと色々な動きをつけられる
Git & GitHub	コードを保存・公開・共同開発できる
フレームワーク	React, Vue などて大きなサイトを作れる
デザイン	もっと見やすく使いやすくできる

付録

もっと詳しく知りたい人向け

box-shadow と rgba()

```
box-shadow: 0 2px 12px rgba(0, 0, 0, 0.08);  
/*      横   縦   ぼかし   色      */
```

- 0 → 横ずれなし、2px → 少し下、12px → ふんわりぼかし
- rgba(0, 0, 0, 0.08) → 黒の8%の濃さ（ほぼ透明な影）
- rgba(赤, 緑, 青, 透明度) — 透明度 0 =透明 ～ 1 =不透明

border の書き方

```
border: 3px solid pink;  
/*      太さ   種類   色      */
```

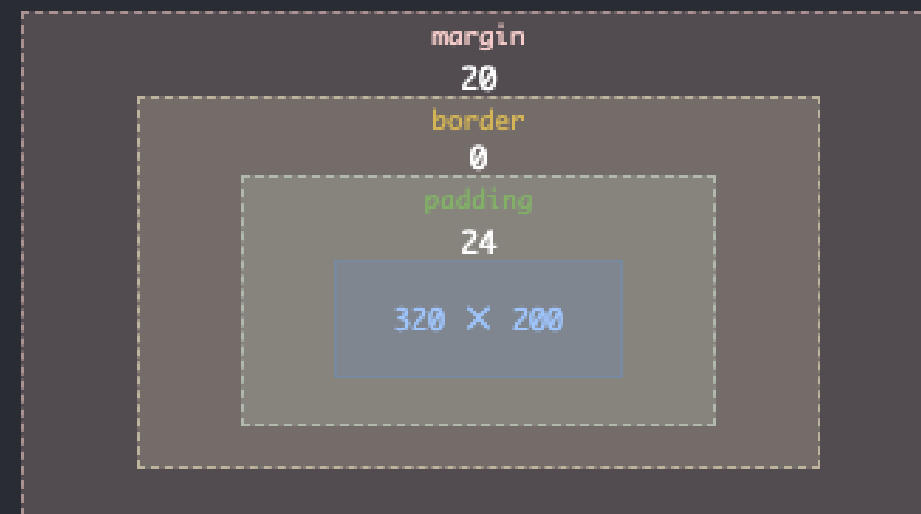
3つの値をスペース区切りで書く。線の種類は `solid`（実線）、`dashed`（破線）、`dotted`（点線）、`none`（なし）。

padding / margin を開発者ツールで確認

ブラウザの **開発者ツール** で、要素の余白を視覚的に確認できます。

F12 でDevToolsを開く →
Elements → 右下の **Computed**
タブ

Chrome DevTools → Elements → Computed タブで確認できる



💡 F12 でDevToolsを開く → Elements → 右下の **Computed** タブ

よく使う HTML タグ一覧

タグ	意味
<code><h1></code> ～ <code><h6></code>	見出し（h1が最大）
<code><p></code>	段落（文章）
<code><div></code>	ブロックのグループ化
<code></code>	リンク
<code></code>	画像（終了タグなし）
<code></code> , <code></code> , <code></code>	リスト
<code><button></code>	ボタン

参考: [MDN HTML要素リファレンス](#)

よく使う CSS プロパティ一覧 (1/2)

プロパティ	意味
<code>color</code>	文字色
<code>background-color</code>	背景色
<code>font-size</code>	文字の大きさ
<code>text-align</code>	文字の揃え方（ <code>center</code> など）
<code>padding</code> / <code>margin</code>	内側 / 外側の余白

よく使う CSS プロパティ一覧 (2/2)

プロパティ	意味
<code>border</code>	枠線（太さ 種類 色）
<code>border-radius</code>	角の丸み
<code>display</code>	表示方法（ <code>flex</code> , <code>block</code> , <code>none</code> ）
<code>box-shadow</code>	影
<code>width</code> , <code>height</code>	幅・高さ

参考: [MDN CSSリファレンス](#)

参考リンク

- [MDN Web Docs](#) — HTML/CSS/JS のリファレンス
- [Can I use](#) — ブラウザの機能対応状況を確認する
- [Google カラーピッカー](#) — カラーコードを選ぶ